



FACULTY OF APPLIED & COMPUTER SCIENCES

DEPARTMENT OF ICT

DIPLOMA: INFORMATION AND COMMUNICATION TECHNOLOGY

SUBJECT : DEVELOPMENT SOFTWARE 3.1
SUBJECT CODE : ASDSX3A
FORMATIVE ASSESSMENT :
DATE : 5 JUNE 2023
DURATION OF TEST : 4 HOURS
EXAMINER(S) : MRS S MOYO
MODERATOR(S) : MR S MHLONGO

REQUIREMENTS : NOT APPLICABLE

INSTRUCTIONS:

1. Download the zipped **YourStudentnumber_Assesment3_2023** NetBeans that contains the GUI for the application to be coded.
2. Unzip the folder and rename the Project (replace the text **YourStudentnumber** with your student number)
3. Open the project from NetBeans and code the program according to the specifications on this question paper.
4. Where variables are declared, use the declared variables for uniformity.
- 5. When you are done coding the application, log onto VUTela and answer the online test that is mostly based on this question paper.**

MARKS: Total = 100
Full marks = 100

THE QUESTION PAPER CONISTS OF: 7 typed pages

DO NOT TURN THE PAGE BEFORE PERMISSION IS GRANTED

You have been requested to create an App for the **Pet Veterinary** in your locality. The Pet veterinary has a policy to give discount to animal owners under certain conditions after a consultation. The Pet Veterinary only allows for dogs and cats. Currently the information on the pets that have been attended to at the Pet Veterinary has been stored in an xml file as shown in **Figure 1**. The manager wants to know information such as the average age of the pets attended to, and the total amount that has been accumulated so far.

Code the application as follows:

- ```
<?xml version="1.0"?>
- <pets>
 - <pet>
 <petName>Max</petName>
 <petType>dog</petType>
 <petBreed>Greyhound</petBreed>
 <age>5</age>
 </pet>
 - <pet>
 <petName>Nelly</petName>
 <petType>cat</petType>
 <petBreed>Havana Brown</petBreed>
 <age>21</age>
 </pet>
 - <pet>
 <petName>Danger</petName>
 <petType>Dog</petType>
 <petBreed>Chihuahua</petBreed>
 <age>8</age>
 </pet>
 - <pet>
 <petName>Sisa</petName>
 <petType>Dog</petType>
 <petBreed>Poodle</petBreed>
 <age>2</age>
 </pet>
+ <pet>
+ <pet>
+ <pet>
+ <pet>
+ <pet>
+ <pet>
</pets>
```



2. Create a parameterless method named **displayPetInfo()** in the **PetDataManagement** form's class and call it in the form's constructor below the call statement for the `parseXMLFile()` method. In the method, write code to display ALL the data from the xml file on the console window as in Figure 3.

```
Output - Assessment22022 (run)

run:
list of pets in the PetsInfo xml file
=====

Pet No: 1
 petName : Max
 petType : dog
 petBreed : Greyhound
 age : 5
Pet No: 2
 petName : Nelly
 petType : cat
 petBreed : Havana Brown
 age : 21
Pet No: 3
 petName : Danger
 petType : Dog
 petBreed : Chihuahua
 age : 8
```

Figure 3

## Section B

### 3. Classes to code

- **PetClass** – should be abstract
  - This class must store the **name** of the pet, the **age**, the **type of pet** (use enum for type of pet) and the **pet breed**.
  - Code one constructor that gets the name, age, type of pet and breed in that order.
  - Code mutator and accessor methods for all instant variables.
  - Validate the name to be 3 characters or more and that the age is >0 but <=25.
  - This application must calculate the amount to be paid for the consultation. The amount is calculated as follows:
    - If age is below 4 years and animal is a dog, the owner gets a discount of 10%.
    - If age is between 4 to 8 years and animal is a dog, the owner gets a discount of 15%.
    - If the age is above 8 years and animal is a dog, the owner gets a discount of 20%.
    - If the age is below 5 and animal is a cat, the owner gets a discount of 9%.
    - If the age is between 5 to 9 years and animal is a cat, the owner gets a discount of 13%.
    - If the age is above 9 years and animal is a cat, the owner gets a discount of 18%.
  - Code an abstract method named **calcConsultationCost()**. Provide the Javadoc comment for the method. The method does not pass any parameter and it returns the amount to be paid by the pet owner for the consultation.
  - Code the toString() method of the PetClass class to display details about the pet as shown

### **DogClass-** inherits PetClass

- Stores the amount paid as an attribute.- code get and set methods for the attribute. The attribute is not passed as a parameter by the constructor. Set this attribute to 0.0 in the constructor. The constructor will be used by the GUI to create an object of this class.
- Code the **calcConsultationCost()** method to calculate the amount to be paid for dog pets consultation(see how the amount should be calculated under **PetClass**).
- Code the **toString()** method to display all the information about the pet including the consultation fee.

### **CatClass-** inherits PetClass

- Stores the amount paid as an attribute.- code get and set methods for the attribute. Code a constructor that inherits the properties of the super class only. The amount paid attribute should not be passed as a parameter so it should be set to 0.0 in this constructor. The constructor will be used by the GUI to create an object of this class.
- Code another constructor for this class th passes all the properties including the amount paid. This constructor will be used to store objects from the database(**petNo not included**).
- Code the **calcConsultationCost()** method to calculate the amount to be paid for cat pets consultation(see how the amount should be calculated under **PetClass**).
- Code the **toString()** method to display all the information about the pet including the consultation fee.

## **4. DA Class code**

Create the folloing methods in the PetDAClass:

- Method **initialisePetDBConnection()** – to be used by the Java application to connect to Test3PetDB.
- A method named **addPet()**- to be used to add dog pets and cat pets objects to the database table tblPets. The method must pass as a parameter the object of the pet to be saved to the database.
- Method **returnCatPets()**- the method returns an arraylist of cat pets only.

- Method **terminateConnection()**- the method terminates the connection established by the **initialisePetDBConnection()** method.
- Call these methods in the **PetClass** class.

## 5. GUI- PetDataManagement form's class

- Group the radio buttons
- Write code for the **actionPerformed** events for **radDog** to hide cmboCatBreed and **radCat** to hide cmboDodBreed. See Figure 4, only the combo box for the cat breeds is visible when the Cat radio button is selected.
- Code the **SavePetActionPerformed event procedure** as follows:
  - Check if the user has entered input in the text fields
    - If any of the text fields has no input, display an error message, otherwise
    - Establish a connection to the database
    - Check which radio button is selected and create an object of the **CatClass** if **radCat** is selected.
    - Call the calcConsultationCost method to calculate the amount paid.
    - Call the set method for the amount paid attribute to assign the calculated amount to the object and
    - call the **addPet** method to add the created cat object to the tblPets otherwise,
    - if **radDog** is selected, create an object of the **DogClass**
    - Call the calcConsultationCost method to calculate the amount paid.
    - Call the set method for the amount paid attribute to assign the calculated amount to the object and
    - call the **addPet** method to add the created dog object to the tblPets and display a message to the user using a message box , once the object has been successfully added to tblPets( **see Figure 4**)
  - Terminate the connection to the database.
- Run the program and use the **Save pet** button to save all the pets that are in the xml file to tblPets.

e. Create the following 4 helper methods for creating the XML Doc:

- method to create a DOM element with text
- method to build a DOM element from a PetClass object
- method to build a DOM element for an array list of Pets
- method to build a DOM document for an array list of pets.

f. Write code for the **Create XML Doc** button to create an xml document of all the pets cats in the database. Only write code to

- Initialise database connection, store pet data into a local array list by calling the **returnCatPets** and **call the helper method that builds the DOM document** and then display the data in the console window in this button. Figure 4 shows the sample out put for the Create XML button.

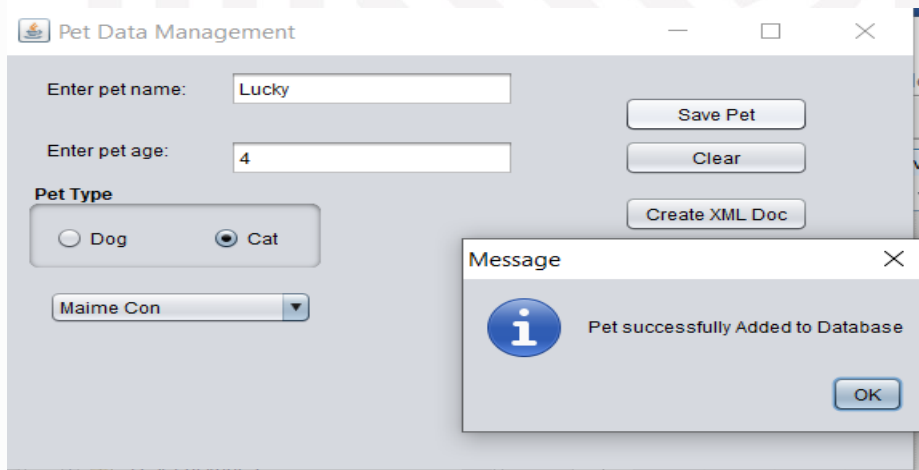


Figure 4

```
Output - Assessment22022 (run)
run:
<?xml version="1.0" encoding="UTF-16"?>
<pets>
 <pet>
 <petName>Nelly</petName>
 <petType>CAT</petType>
 <petBreed>Havana Brown</petBreed>
 <age>21</age>
 <amountPaid>297.0</amountPaid>
 </pet>
 <pet>
 <petName>Danger</petName>
 <petType>CAT</petType>
 <petBreed>Chihuahua</petBreed>
 <age>8</age>
 <amountPaid>323.0</amountPaid>
 </pet>
 <pet>
 <petName>Lucky</petName>
 <petType>CAT</petType>
 <petBreed>Maime Con</petBreed>
 <age>4</age>
 <amountPaid>318.5</amountPaid>
 </pet>
</pets>
```

Figure 5

\*\*\*\*\*The End \*\*\*\* Buena Suerte\*\*\*\*\*